

IME 775 – Lecture 13

Function Approximation: Target Functions, Perceptrons, and Logic Gates

1. From Probability to Function Approximation

Chapters 1–6 built the mathematical toolkit: linear algebra, calculus, probability, and Bayesian methods. Now we put them to work. The central question of neural networks is:

Given sample input-output pairs, can we build a machine that approximates the unknown function mapping inputs to outputs?

Neural networks answer "yes" by providing a **unified framework** to model an extremely wide variety of arbitrarily complicated functions — without ever knowing the function in closed form.

Key idea: Intelligence $\approx$ function evaluation. If we can model the right function, we can automate the intelligent task.

2. Neural Networks: The 30,000-Foot View

Traditional computing (von Neumann architecture) separates the processor from the program. Neural networks are fundamentally different: there is no separate program — the network *is* the computation.

Two Levers of Control

Lever	What it controls	When it is set
Architecture	Number of neurons, connections between them	Chosen before training (based on problem type)
Parameter values	Weights of connections	Learned during training

Expressive Power

The variety and complexity of functions a neural network can represent is its **expressive power**. It increases with:

- More neurons
- More connections between neurons
- More layers

The more complex the target function, the more expressive power is needed.

Supervised vs. Unsupervised Learning

Type	Training data	Example
Supervised	$\langle \text{input, desired output} \rangle$ pairs (manually labeled)	Image classification, spam detection
Unsupervised	Inputs only (no labels)	Clustering, dimensionality reduction
Semi-supervised	Small fraction labeled, rest unlabeled	Label-efficient learning
Self-supervised	Labels created programmatically from the data itself	Language model next-word prediction, contrastive learning

Nuanced point — why supervised learning dominates in practice:

Supervised learning has a clear optimization target (match the ground truth), making training straightforward. Unsupervised and self-supervised methods are active research areas because labeling is expensive — a single ImageNet dataset

required millions of human annotations. Self-supervised learning (e.g., masked language modeling in BERT) has recently closed the gap by creating labels *from* the data, avoiding manual annotation entirely.

Critical insight: Neither training nor architecture selection requires knowing the target function in closed form. We only need sample input-output pairs. This is what makes neural networks practical — the underlying function for most real-world problems is unknown.

3. Target Functions: Modeling Real-World Problems

Most real-world intelligent tasks can be expressed as functions $y = f(\vec{x})$:

- **Inputs** $\vec{x}$: quantifiable variables (pixel values, sensor readings, prices)
- **Outputs** y : the decision or prediction

3.1 Logical Functions

Inputs and outputs are **binary** (0 or 1). These are often placed on top of other models to combine their decisions.

Logical OR — The Timid Cat:

A cat runs away from anything hard OR sharp. Two separate models output binary hardness and sharpness decisions. The OR gate combines them:

Hardness (x_0)	Sharpness (x_1)	Run away (y)
0	0	0
0	1	1
1	0	1
1	1	1

Logical AND — The Less Timid Cat:

This cat only runs from things that are BOTH hard AND sharp:

x_0	x_1	y
0	0	0
0	1	0
1	0	0
1	1	1

Logical XOR — Friend Recommendation:

Two people should be recommended as friends if they BOTH like rock music or BOTH dislike it (agreement matters):

Person A likes rock (x_0)	Person B likes rock (x_1)	Recommend (y)
0	0	1
0	1	0
1	0	0
1	1	1

This is $y = \overline{x_0 \oplus x_1}$ (NOT XOR). We will see later that XOR is the *simplest* function a single perceptron cannot model.

m-out-of-n Trigger — Face Detection with Occlusion:

Separate detectors find noses, eyes, lips, ears. If *any* 2 are detected, declare a face. This robustness against occlusion (objects blocking the camera's view) is critical in computer vision.

3.2 Classifier Functions

A classifier outputs a **categorical** variable — it assigns an input to one of a finite set of classes.

$$q(\vec{x}) = \begin{cases} 0 & \text{not a face} \\ 1 & \text{face} \end{cases}$$

Discriminative vs. Generative classifiers:

Type	Output	Example
Discriminative	Category label (0 or 1)	"This is a face"
Generative	Probability $P(\text{face} \mid \vec{x}) \in [0, 1]$	"87% chance this is a face"

Generative classifiers output continuous probabilities, which are more informative than hard category labels.

3.3 General Functions

Some problems require **continuous** output:

- Speed estimation for a self-driving car
- Stock price prediction
- Temperature forecasting

These are regression problems where $y \in \mathbb{R}$.

4. Decision Boundaries: The Geometric View

A classifier partitions the input space into regions, each corresponding to a class. The boundary between regions is called the **decision boundary**.

Linear vs. Nonlinear Decision Boundaries

Consider the cat brain with two continuous inputs: hardness (x_0) and sharpness (x_1).

- The **true decision boundary** is typically a curved surface (nonlinear)
- A **linear approximation** (straight line / hyperplane) is simpler but introduces misclassification errors
- Points between the true and approximate boundaries are wrongly classified

Nuanced point — linear boundaries are both a strength and a limitation:

Linear decision boundaries (hyperplanes) are easy to represent, optimize, and interpret. But they fail on inherently nonlinear problems. The entire progression from perceptrons to deep networks is driven by the need for *nonlinear* decision boundaries. A single perceptron can only produce linear boundaries — this is its fundamental limitation.

Significance of Sign

For a surface $q(\vec{x}) = \vec{w}^T \vec{x} + b = 0$:

Condition	Geometric meaning
$\vec{w}^T \vec{x} + b = 0$	$\vec{x}$ lies on the boundary
$\vec{w}^T \vec{x} + b > 0$	$\vec{x}$ is on the positive side
$\vec{w}^T \vec{x} + b < 0$	$\vec{x}$ is on the negative side

This sign-based partitioning is exactly what a perceptron exploits for classification.

Workout: The decision boundary is $2x_0 + 3x_1 - 6 = 0$. Classify the points $\vec{a} = (1, 2)^T$ and $\vec{b} = (4, 0)^T$.

Solution:

- For $\vec{a}$: $2(1) + 3(2) - 6 = 2 + 6 - 6 = 2 > 0 \rightarrow$ positive side (class 1)
- For $\vec{b}$: $2(4) + 3(0) - 6 = 8 - 6 = 2 > 0 \rightarrow$ positive side (class 1)

Both points are on the same side of the boundary.

Training Data and Decision Boundaries

In practice, we do not know the true regions. We have **training data**: sampled $\langle \text{input}, \text{ground truth} \rangle$ pairs. The decision boundary is optimized to classify training points correctly.

Good training data spans the true class regions → good decision boundary. **Bad training data** clusters in small subregions → poor generalization.

5. The Heaviside Step Function

The Heaviside step function (or simply the **step function**) is the activation function of the perceptron:

$$\theta(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

It converts any real number into a binary decision: negative inputs map to 0, non-negative inputs map to 1.

Deeper explanation:

The step function is the simplest possible nonlinearity — it is what makes the perceptron a classifier rather than a linear function. Without it, the perceptron would just compute $\vec{w}^T \vec{x} + b$, which is a continuous value. The step function "snaps" this continuous value to a binary decision. Later in the course (Ch. 8), we will replace it with smoother activations (sigmoid, ReLU) that allow gradient-based training.

6. Hyperplanes as Decision Boundaries

Recall from Chapter 2: for a fixed weight vector $\vec{w}$ and bias b , the equation

$$\vec{w}^T \vec{x} + b = 0$$

defines a **hyperplane** in the input space.

Why $\vec{w}$ is Normal to the Hyperplane

Take any two points $\vec{x}_0, \vec{x}_1$ on the hyperplane:

- $\vec{w}^T \vec{x}_0 + b = 0$
- $\vec{w}^T \vec{x}_1 + b = 0$

Subtracting: $\vec{w}^T (\vec{x}_1 - \vec{x}_0) = 0$

Since $(\vec{x}_1 - \vec{x}_0)$ is any vector lying on the hyperplane, $\vec{w}$ is **perpendicular to all directions on the hyperplane** — it is the normal vector.

Examples by Dimension		
Input dim	Decision boundary	Example
1D	Point	$w_0 x_0 + b = 0 \rightarrow$ a threshold
2D	Line	$w_0 x_0 + w_1 x_1 + b = 0$
3D	Plane	$w_0 x_0 + w_1 x_1 + w_2 x_2 + b = 0$
d-D	Hyperplane	$\vec{w}^T \vec{x} + b = 0$

Workout: For the hyperplane $x_0 + x_1 + x_2 = 0$ in 3D, what is $\vec{w}$? What is b ? Which side is the point $(1, 1, -1)^T$ on?

Solution:

- $\vec{w} = (1, 1, 1)^T, b = 0$
- Evaluate: $1 + 1 + (-1) = 1 > 0 \rightarrow$ positive side

7. The Perceptron

The perceptron combines the step function with a hyperplane into a single computational unit:

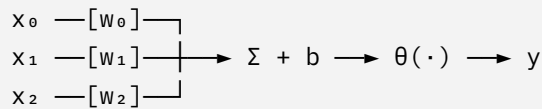
$$P(\vec{x}) = \theta(\vec{w}^T \vec{x} + b)$$

What It Does

1. Compute the weighted sum: $z = \vec{w}^T \vec{x} + b = \sum_i w_i x_i + b$
2. Apply the step function: output 1 if $z \geq 0$, else output 0

This makes the perceptron a **linear binary classifier**: it maps all points on one side of the $(\vec{w}, b)$ hyperplane to 0 and all points on the other side to 1.

The Perceptron Diagram



Each input x_i is multiplied by weight w_i , the products are summed with the bias b , and the step function produces a binary output.

Nuanced point — what the perceptron cannot do:

A single perceptron can only produce a **linear** (hyperplanar) decision boundary. This means it can only classify data that is **linearly separable** — where a single hyperplane can perfectly separate the classes. Many real-world problems are not linearly separable (e.g., XOR), which is why we need multilayer networks.

Workout: A perceptron has $\vec{w} = (2, -1)^T$ and $b = 1$. Compute the output for inputs $\vec{x}_1 = (1, 1)^T$, $\vec{x}_2 = (0, 3)^T$, and $\vec{x}_3 = (-1, 0)^T$.

Solution:

- $P(\vec{x}_1) = \theta(2(1) + (-1)(1) + 1) = \theta(2) = 1$
- $P(\vec{x}_2) = \theta(2(0) + (-1)(3) + 1) = \theta(-2) = 0$

- $P(\vec{x}_3) = \theta(2(-1) + (-1)(0) + 1) = \theta(-1) = 0$

Decision boundary: $2x_0 - x_1 + 1 = 0$, i.e., $x_1 = 2x_0 + 1$.

8. Modeling Logic Gates with Perceptrons

8.1 Perceptron for AND

$$y = \theta(x_0 + x_1 - 1.5)$$

Weights: $w_0 = 1, w_1 = 1$. Bias: $b = -1.5$.

x_0	x_1	$z = x_0 + x_1 - 1.5$	$y = \theta(z)$
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	0.5	1

The decision boundary $x_0 + x_1 = 1.5$ separates the single $(1, 1)$ point from the rest.

8.2 Perceptron for OR

$$y = \theta(x_0 + x_1 - 0.5)$$

Weights: $w_0 = 1, w_1 = 1$. Bias: $b = -0.5$.

x_0	x_1	$z = x_0 + x_1 - 0.5$	$y = \theta(z)$
0	0	-0.5	0
0	1	0.5	1
1	0	0.5	1
1	1	1.5	1

The decision boundary $x_0 + x_1 = 0.5$ separates $(0, 0)$ from the rest.

8.3 Perceptron for NOT

$$y = \theta(-x_0 + 0.5)$$

Weight: $w_0 = -1$. Bias: $b = 0.5$.

x_0	$z = -x_0 + 0.5$	$y = \theta(z)$
0	0.5	1
1	-0.5	0

Geometric Interpretation

Each logic gate corresponds to a specific **placement of the decision line** on the 2D input plane:

Gate	Decision boundary	Separates
AND	$x_0 + x_1 = 1.5$	Only $(1, 1)$ on the "1" side
OR	$x_0 + x_1 = 0.5$	Only $(0, 0)$ on the "0" side
NOT	$x_0 = 0.5$	Flips the input

Key insight: AND and OR use the same weights $(1, 1)$ but different biases. The bias controls *where* the decision boundary sits. A high threshold (AND, $b = -1.5$) requires both inputs to be 1; a low threshold (OR, $b = -0.5$) requires only one.

Workout: Design a perceptron for a 3-input AND gate: $y = x_0 \wedge x_1 \wedge x_2$.

Solution:

- We need $y = 1$ only when all three inputs are 1, i.e., $x_0 + x_1 + x_2 \geq 2.5$
- Weights: $w_0 = w_1 = w_2 = 1$, Bias: $b = -2.5$
- $y = \theta(x_0 + x_1 + x_2 - 2.5)$

Verify: $(1, 1, 1) \rightarrow \theta(0.5) = 1$. $(1, 1, 0) \rightarrow \theta(-0.5) = 0$. ✓

Workout: Design a perceptron for a 2-of-3 majority vote: output 1 if at least 2 of the 3 binary inputs are 1.

Solution:

- We need $x_0 + x_1 + x_2 \geq 2$, so threshold at 1.5
- Weights: $w_0 = w_1 = w_2 = 1$, Bias: $b = -1.5$
- $y = \theta(x_0 + x_1 + x_2 - 1.5)$

Verify: $(1, 1, 0) \rightarrow \theta(0.5) = 1$. $(1, 0, 0) \rightarrow \theta(-0.5) = 0$. ✓

9. Why XOR Breaks the Perceptron

The logical XOR has this truth table:

x_0	x_1	$y = x_0 \oplus x_1$
0	0	0
0	1	1
1	0	1
1	1	0

The points $(0, 0)$ and $(1, 1)$ map to 0, while $(0, 1)$ and $(1, 0)$ map to 1. These are interleaved on the plane — no single straight line can separate the 0s from the 1s.

Geometric proof: The class-0 points are at opposite corners of the unit square, as are the class-1 points. Any line that puts $(0, 1)$ and $(1, 0)$ on one side must also include at least one of $(0, 0)$ or $(1, 1)$. Therefore, XOR is **not linearly separable**.

This was a famous limitation discovered in 1969 (Minsky & Papert), which temporarily discouraged neural network research. The solution — multilayer perceptrons — is the subject of Lecture 14.

Key Takeaways

1. **Target functions** express real-world problems as $y = f(\vec{x})$ — logical, classifier, or general
 2. **Decision boundaries** partition the input space into class regions; their shape determines classifier complexity
 3. The **Heaviside step function** $\theta(x)$ converts continuous values to binary decisions
 4. The **perceptron** $P(\vec{x}) = \theta(\vec{w}^T \vec{x} + b)$ is a linear binary classifier
 5. A single perceptron can model AND, OR, NOT — but **not XOR**
 6. XOR's failure proves that linear decision boundaries are insufficient for many problems → motivation for MLPs
 7. **Expressive power** = architecture + parameter values; more neurons and connections = more complex functions
-

PyTorch Connection

```
import torch

def heaviside(x):
    return (x >= 0).float()

def perceptron(X, w, b):
    z = X @ w + b
    return heaviside(z)

# AND gate
```

```

X = torch.tensor([[0, 0], [0, 1], [1, 0], [1, 1]],
dtype=torch.float)
w_and = torch.tensor([1.0, 1.0])
b_and = torch.tensor(-1.5)
print("AND:", perceptron(X, w_and, b_and)) # [0, 0, 0, 1]

# OR gate
w_or = torch.tensor([1.0, 1.0])
b_or = torch.tensor(-0.5)
print("OR: ", perceptron(X, w_or, b_or)) # [0, 1, 1, 1]

# NOT gate (single input)
X_not = torch.tensor([[0], [1]], dtype=torch.float)
w_not = torch.tensor([-1.0])
b_not = torch.tensor(0.5)
print("NOT:", perceptron(X_not, w_not, b_not)) # [1, 0]

# Attempting XOR with a single perceptron – IMPOSSIBLE
# No w, b exists such that perceptron(X, w, b) = [0, 1, 1, 0]

```